MCP, documentos fuente y contratos de datos en Acelerador

Documento consolidado a partir de la documentacion previa existente en la raiz del proyecto, `mcp-server/`, `docs/`, `acelerador_panel/backend/` y `acelerador_evaluador_ANECA/`.

Autor: Basilio Lagares

Objetivo editorial de esta version:

- conservar lo que ya era valido,

ordenar lo que estaba disperso,

- distinguir entre implementado, parcial y previsto,

y dejar una base reutilizable para el equipo.

1. Inventario inicial de material encontrado

Documentacion previa directa sobre MCP y tratamiento documental

- **`MCP_Documentacion.md`**

`MCP_Documentacion.docx`

Nota: el archivo tiene extension `.docx`, pero en el repositorio actual contiene texto plano y replica el contenido historico en formato markdown, no un DOCX binario valido.

- `README.md`
-
- **Documentacion tecnica asociada al modulo MCP**

`mcp-server/README.md`

- **`mcp-server/REGISTRO_TECNICO_2026-03-23.md`**

`mcp-server/TESTS_RESULTS.md`

- **`mcp-server/resultados/validacion_extract_pdf_report.md`**

`mcp-server/resultados/validacion_ocr_agresiva_report.md`

Documentacion de estado y arquitectura general del proyecto

- `docs/2026-03-24-resumen-trabajo.md`

`docs/estado-tecnico-mvp.md`

- `docs/control-versiones-estado-tecnico-mvp.md`

`docs/estrategia-testing-mvp.md`

- `docs/2026-03-25-checkpoint-backend-tutorias.md`
-
- ## Documentacion del backend actual y su relacion futura con MCP

`acelerador_panel/backend/README.md`

- `acelerador_panel/backend/docs/02-api-rest-contratos.md`

`acelerador_panel/backend/docs/03-integracion-bd-frontend.md`

- `acelerador_panel/backend/docs/04-operacion-validacion-y-mcp.md`
-
- ## Codigo relevante del modulo MCP

`mcp-server/extract_pdf.php`

- `mcp-server/server.php`

`mcp-server/worker_jobs.php`

- `mcp-server/tests/unit_extract_pdf.php`

`mcp-server/test_extract_pdf.ps1`

- `mcp-server/test_extract_pdf_ocr_aggressive.ps1`

`mcp-server/resultados/fuente_db_config.json`

- `mcp-server/resultados/fuente_db_config_aneca.example.json`

`mcp-server/resultados/fuente_db_config_dialnet.example.json`

- `mcp-server/resultados/resultados.json`

`mcp-server/resultados/jobs/...`

Codigo relevante del flujo documental ANECA ya integrado en producto

- `acelerador_evaluador_ANECA/src/Pipeline.php`

`acelerador_evaluador_ANECA/src/TextCleaner.php`

- `acelerador_evaluador_ANECA/src/AnecaExtractor.php`

`acelerador_evaluador_ANECA/src/OcrProcessor.php`

- **`acelerador_evaluador_ANECA/tests/unit_src.php`**

`acelerador_evaluador_ANECA/evaluador_aneca_*/procesar_pdf.php`

- **`acelerador_evaluador_ANECA/evaluador_aneca_*/guardar_evaluacion.php`**

`acelerador_evaluador_ANECA/evaluador_aneca_*/guardar_complemento.php`

- **`acelerador_evaluador_ANECA/evaluador_aneca_*/schema.sql`**

`acelerador_evaluador_ANECA/output/json/*.json`

Codigo relevante de pipelines paralelos o heredados

- **`meritos/scraping/src/Pipeline.php`**

`meritos/scraping/src/PdfToImage.php`

- **`meritos/scraping/src/OcrProcessor.php`**

`meritos/scraping/src/AnecaExtractor.php`

- **`meritos/scraping/public/subir.php`**
-
- ### Evidencias adicionales de arquitectura real

`acelerador_panel/backend/public/index.php`

- **`acelerador_panel/fronten/lib/tutor_grupos_service.php`**

ramas detectadas en git: `Desarrollo`, `main`, remotas `Backend`, `Frontend`, `Sql`

2. Documento final unificado

2.1 Objetivo

Esta documentacion existe para aclarar como se esta tratando hoy la informacion procedente de documentos fuente en Acelerador, cual es el papel real de MCP dentro del repositorio, que contratos JSON ya se estan usando de facto y como encajan estas piezas con la arquitectura actual del backend y del frontend.

No pretende presentar una arquitectura idealizada. Pretende dejar trazabilidad util para el equipo:

- de lo que ya funciona,

de lo que esta separado pero preparado para integrarse,

- de lo que sigue siendo una linea de evolucion,

y de los puntos donde conviene converger sin perder trabajo previo.

2.2 Contexto del problema

Acelerador necesita convertir informacion heterogenea en datos estructurados que puedan ser evaluados, almacenados y presentados sin depender del formato original de cada fuente.

Ese problema aparece en varias capas del proyecto:

- un PDF puede venir como texto nativo, como imagen escaneada o como mezcla;

un documento puede contener informacion util, pero no en una forma directamente consumible por el backend;

- una misma evidencia puede terminar siendo usada por distintos consumidores: extraccion tecnica, evaluacion ANECA, backend de negocio o frontend;

y a futuro no todas las fuentes van a ser documentos: tambien aparecen previstas fuentes externas tipo ANECA, Dialnet u otras APIs/servicios.

Por eso no conviene acoplar la logica de negocio al formato de origen. El backend no deberia tener que interpretar directamente un PDF, ni el frontend deberia conocer las peculiaridades de cada extractor. Lo sostenible es introducir una capa intermedia que:

- absorba la heterogeneidad de entrada,

normalice el contenido,

- y exponga una salida estructurada y estable.
-
- ## 2.3 Que es MCP en este proyecto
-
- En Acelerador, MCP no aparece como una teoria abstracta, sino como un modulo concreto separado en `mcp-server/` cuya funcion actual es servir de capa intermedia entre fuentes heterogeneas y consumidores internos.
-
- Su papel real hoy es este:

recibir una fuente de datos (`pdf` o `db`);

- extraer texto o contenido util;

aplicar una transformacion a un contrato JSON fijo;

- exponerlo por CLI o por HTTP;

y gestionar procesamiento sincronico o asincronico segun el tamano del PDF.

Es importante distinguir dos cosas:

- `mcp-server/` si esta implementado y probado como modulo autonomo de extraccion y normalizacion generica.

Su integracion completa con el resto de Acelerador todavia no es la ruta unica del producto.

En paralelo, el backend modular de tutorias en `acelerador_panel/backend/` ya funciona sin depender de MCP y explicita ese desacoplamiento mediante un punto de extension futuro (`AssignmentEventPublisherInterface` + `NullAssignmentEventPublisher`).

Conclusión practica: en este repositorio MCP ya existe como pieza arquitectonica intermedia, pero aun no es el unico canal de entrada documental ni la unica frontera de integracion entre fuente y negocio.

2.4 Fuentes de entrada

PDFs de texto

Son la fuente mejor soportada en el estado actual.

Rutas reales detectadas:

- `mcp-server/extract_pdf.php`

`acelerador_evaluador_ANECA/src/Pipeline.php`

Comportamiento actual:

- en `mcp-server`, la extraccion nativa se hace con `smalot/pdfparser`;

en `acelerador_evaluador_ANECA`, la extraccion se hace con `pdftotext`.

PDFs hibridos o escaneados

Aplican de forma parcial, con diferencias importantes segun el modulo.

Estado actual por modulo:

- `mcp-server`: soportado mediante OCR con `pdftoppm` + `tesseract` cuando el texto nativo es insuficiente.

`meritos/scraping`: existe pipeline OCR basado en conversion a imagen + Tesseract.

- `acelerador_evaluador_ANECA`: no hay fallback OCR conectado al pipeline principal actual; el `Pipeline` usa `pdftotext` y falla si no obtiene texto util.
-
- **Otros documentos fuente**
-
- No se ha encontrado en el repositorio una familia adicional de adaptadores documentales ya formalizada mas alla de PDF.
-
- Si se habla de "documentos fuente", hoy el codigo real apunta sobre todo a PDF y a texto procedente de base de datos.
-
- **Fuentes externas o no documentales previstas**
-
- Si aparecen en la documentacion y configuracion futura:

ANECA

- Dialnet

criterios de orquestacion por `ORCID`, `DOI` y `rama`

Evidencias reales:

- `mcp-server/resultados/fuente_db_config_aneca.example.json`

`mcp-server/resultados/fuente_db_config_dialnet.example.json`

- `mcp-server/REGISTRO_TECNICO_2026-03-23.md`

`docs/2026-03-24-resumen-trabajo.md`

No se ha encontrado implementacion final de una integracion externa multi-fuente por reglas de negocio. Eso sigue en fase de diseno/preparacion.

PDDs

No se han encontrado referencias explicitas a `PDD` o `PDDs` en documentacion ni en codigo del repositorio actual. El termino practico que si aparece y esta implementado es `PDF`.

2.5 Flujo general de tratamiento de la informacion

No hay un unico pipeline en produccion dentro del repo. Hoy conviven al menos dos rutas reales y una ruta heredada/paralela.

Flujo real del modulo MCP

`fuente pdf|db -> extraccion de texto -> normalizacion minima -> extraccion de campos -> contrato JSON fijo -> consumidor`

Mas en detalle:

1. Entrada:
- CLI legacy o por flags

HTTP `POST /extract-pdf`

- HTTP `POST /extract-data`
1. Diagnostico de PDF:
- tamano

numero de paginas

- disponibilidad OCR
1. Extraccion:
- nativa para PDF de texto

OCR si el texto nativo no es suficiente

- lectura por PDO si la fuente es `db`

1. Transformacion:
- `DocumentoExtractor` aplica regex y produce el contrato de salida
1. Entrega:
- respuesta directa

o cola async + `job_id` + consulta por `GET /jobs/{job_id}`

Flujo real del evaluador ANECA

`subida PDF -> pdftotext -> limpieza -> extraccion academica -> JSON estructurado -> opcion de completado manual -> persistencia en BD -> vistas de evaluacion`

Mas en detalle:

2. El usuario sube un PDF desde `evaluador_aneca_*`.

`Pipeline.php` extrae texto con `pdftotext`.

3. `TextCleaner` limpia espacios y saltos.

`AnecaExtractor` genera un JSON academico por bloques.

4. El JSON se muestra, se puede guardar directamente o completar manualmente.

El JSON final se almacena en `json_entrada` dentro de la tabla `evaluaciones`.

Flujo heredado/paralelo en `meritos/scraping`

`pdf -> imagenes -> OCR -> limpieza -> extraccion -> json`

Existe, pero no es la ruta mejor alineada con el estado actual del producto y no comparte el mismo contrato que `acelerador_evaluador_ANECA`.

2.6 Tratamiento especifico de PDFs / documentos fuente

Tratamiento actual en `mcp-server`

Archivo principal:

- `mcp-server/extract_pdf.php`
-
- Capacidades observadas:

soporte de PDF nativo con `smalot/pdfparser`;

- soporte OCR con `pdftoppm` + `tesseract`;

deteccion de disponibilidad de binarios locales o globales;

- limites configurables:

`MAX_OCR_PAGES`

- `OCR_BATCH_SIZE`

`MAX_SYNC_PDF_BYTES`

- `MAX_SYNC_PAGES`

estrategia sync/async segun diagnostico;

- soporte de procesamiento por cola con `worker_jobs.php`.
-
- Limitaciones:

el contrato de salida actual es generico y pequeno; sirve para documentos tipo factura/prueba, no para evaluacion academica rica;

- el valor del modulo hoy esta mas en el desacoplamiento de fuentes y en la infraestructura de extraccion que en la riqueza semantica del JSON final.
-
- **Tratamiento actual en `acelerador_evaluador_ANECA`**
-
- Archivos principales:

`acelerador_evaluador_ANECA/src/Pipeline.php`

- `acelerador_evaluador_ANECA/src/AnecaExtractor.php`
-
- Capacidades observadas:

extrae texto con `pdftotext`;

- limpia texto;

genera un JSON academico por bloques;

- guarda el JSON en `output/json/`;

adjunta `archivo_pdf`, `json_generado` y `texto_extraido`.

Limitaciones reales:

- depende de una ruta Windows fija a `pdftotext.exe`;

no integra hoy el `OcrProcessor` del mismo modulo;

- por tanto, los PDF escaneados o sin capa de texto pueden no ser tratables por esta ruta principal;

el extractor usa heuristicas por lineas y deja muchos campos en revision manual, lo cual es coherente con el dominio, pero conviene explicitarlo.

Tratamiento actual en `meritos/scraping`

Archivos principales:

- `meritos/scraping/src/Pipeline.php`

`meritos/scraping/src/PdfToImage.php`

- `meritos/scraping/src/OcrProcessor.php`
-
- Capacidades observadas:

convierte PDF a imagen con `pdftoppm`;

- ejecuta OCR con Tesseract;

limpia texto y genera JSON.

Limitaciones:

- el extractor final de este flujo no devuelve el contrato academico rico, sino el contrato simple tipo factura;

por eso debe leerse hoy como un pipeline paralelo o heredado, no como la base documental consolidada del evaluador ANECA.

2.7 Normalizacion de datos y contrato de intercambio

La idea de fondo que si se repite en varias partes del proyecto es esta:

`fuente heterogenea -> estructura comun en JSON -> consumidor`

Eso desacopla responsabilidades:

- el extractor se ocupa del origen y del preprocesado;

el backend de negocio consume una estructura ya normalizada;

- el frontend trabaja con datos estructurados, no con PDF bruto.
-
- **Papel de JSON en el proyecto**
-
- JSON ya se esta usando de dos maneras reales:
-

1. Como contrato de salida tecnico del MCP
- en CLI;

en HTTP;

- en resultados de jobs;

en archivos de resultado.

2. Como contrato intermedio de trabajo en ANECA
- el JSON extraido pasa entre pantalla de procesamiento, formularios de completado, evaluacion y persistencia;

se guarda en BD como `json_entrada`;

- puede fusionarse con datos manuales antes de evaluar.
-
- **Contrato entre productor y consumidor**
-
- El concepto ya esta presente aunque todavia no exista un `JSON Schema` formal versionado:

el productor es quien extrae y normaliza;

- el consumidor es quien evalua, persiste, integra o presenta;

ambos deben acordar una estructura estable para evitar acoplamientos fragiles.

Hoy hay dos contratos "de hecho":

- contrato MCP simple y fijo;

contrato ANECA academico, mas rico y orientado a evaluacion.

Lo que falta es el cierre formal de esos contratos mediante versionado y schema compartido.

2.8 Estructuras de salida detectadas

A. Contrato detectado en `mcp-server`

Contrato fijo verificado en codigo y tests:

```
{
  "tipo_documento": "FACTURA | null",
  "numero": "string | null",
  "fecha": "string | null",
  "total_bi": "string | null",
  "iva": "string | null",
  "total_a_pagar": "string | null",
  "texto_preview": "string"
}
```

Observaciones:

- las 7 claves existen siempre;

`texto_preview` es obligatorio y siempre string;

- el resto puede venir a `null`;

`detectarCamposNegocioFaltantes()` calcula los campos vacios o ausentes;

- este contrato esta validado por `mcp-server/tests/unit_extract_pdf.php`.
-
- Ejemplo real encontrado en `mcp-server/resultados/jobs/job_20260323_110834_ce1820c8/result.json`:
-
- {
- "tipo_documento": null,
- "numero": null,
- "fecha": null,

```
    "total_bi": null,
    "iva": null,
    "total_a_pagar": null,
    "texto_preview": "SUPER ..."
  }
```

Cuando se consume por API, suele venir envuelto en una respuesta tipo:

```
  {
    "ok": true,
    "queued": false,
    "faltantes": ["numero", "fecha"],
    "resultado": {
      "tipo_documento": null,
      "numero": null,
      "fecha": null,
      "total_bi": null,
      "iva": null,
      "total_a_pagar": null,
      "texto_preview": "..."
    }
  }
```

B. Contrato detectado en `acelerador_evaluador_ANECA`

Estructura base observada:

```
  {
    "bloque_1": {
      "publicaciones": [],
      "libros": [],
      "proyectos": [],
      "transferencia": [],
      "tesis_dirigidas": [],
      "congresos": [],
      "otros_meritos_investigacion": []
    },
    "bloque_2": {
      "docencia_universitaria": [],
      "evaluacion_docente": [],
      "formacion_docente": [],
      "material_docente": []
    },
```

```
•     "bloque_3": {
•       "formacion_academica": [],
•       "experiencia_profesional": []
•     },
•     "bloque_4": [],
•     "metadatos_extraccion": {
•       "comite": "CSYJ",
•       "subcomite": null,
•       "archivo_pdf": null,
•       "fecha_extraccion": "2026-04-01T20:38:07+02:00",
•       "version_esquema": "1.0",
•       "requiere_revision_manual": true
•     },
•     "archivo_pdf": "exp_xxx.pdf",
•     "json_generado": "exp_xxx.json",
•     "texto_extraido": "..."
•   }
•
```

- Ejemplo real:

`acelerador_evaluador_ANECA/output/json/exp_69cd660f626b5.json`

Campos y comportamiento observados:

- top-level:

`bloque_1`, `bloque_2`, `bloque_3`, `bloque_4`: esperables en toda salida generada

- `metadatos_extraccion`: presente

`archivo_pdf`, `json_generado`, `texto_extraido`: anadidos por `Pipeline.php`

- items internos:

muchos campos son opcionales y dependen del tipo de evidencia

- se repiten patrones como `fuente_texto`, `confianza_extraccion`, `requiere_revision`

el extractor esta orientado a producir evidencias preliminares, no una verdad final cerrada

C. JSON como contrato intermedio de evaluacion

En ANECA el JSON no termina en la extraccion:

- `procesar_pdf.php` lo pasa a formularios ocultos;

`guardar_evaluacion.php` lo persiste tal cual;

- `guardar_complemento.php` fusiona el JSON extraido con datos manuales;

la tabla `evaluaciones` guarda ese `json_entrada` como evidencia base.

Esto confirma que JSON ya es, en la practica, el contrato entre productor documental y consumidor de negocio.

2.9 Estado actual de implementacion

Implementado

- Modulo MCP autonomo para:

PDF nativo

- OCR

fuente DB

- API HTTP

cola async

- Contrato JSON fijo del `mcp-server`

Tests unitarios de `mcp-server` en PASS (`13/13`)

- Pipeline ANECA de:

PDF con texto

- limpieza

extraccion academica

- guardado JSON

persistencia en `json_entrada`

- Flujos de completado manual y fusion JSON en evaluadores ANECA

Backend modular de tutorias desacoplado de MCP, con contrato REST propio y punto de extension preparado

Parcial / en progreso

- Soporte uniforme de OCR en todos los flujos documentales

Convergencia entre `mcp-server`, `acelerador_evaluador_ANECA` y `meritos/scraping`

- Migracion del frontend del panel para consumir API en lugar de SQL directo

Formalizacion transversal de contratos entre modulos

Pendiente de integracion

- Integrar MCP como capa comun real entre todas las fuentes heterogeneas y los consumidores de negocio

Parametrizar la extraccion ANECA por area/comite de manera coherente

- Sustituir flujos paralelos por una estrategia unificada o, al menos, documentar oficialmente cual es la ruta canonica
-
- **Previsto a futuro**
-

Orquestacion por `ORCID`, `DOI` y `rama`

- Conexion con fuentes externas tipo ANECA, Dialnet u otras

`JSON Schema` formal de entrada/salida

- Expansión de MCP como capa intermedia comun para integraciones nuevas
-
- **2.10 Relacion entre MCP y la arquitectura actual**
-
- La relacion real hoy es progresiva y por capas.
-
- **MCP respecto al backend actual**
-
- El backend de `acelerador_panel/backend` se ha construido primero para validar funcionalidad basica sin bloquearse por la integracion documental.
-
- Evidencia:

`docs/2026-03-25-checkpoint-backend-tutorias.md`

- `acelerador_panel/backend/docs/04-operacion-validacion-y-mcp.md`
-
- Decision arquitectonica visible:

backend funcionando sin MCP;

- extension futura prevista mediante `AssignmentEventPublisherInterface`;

implementacion actual nula: `NullAssignmentEventPublisher`.

Esto encaja con la idea descrita por el equipo:

- primero validar el backend real;

despues integrar MCP sin forzar dependencias prematuras.

MCP respecto al frontend actual

La integracion frontend -> backend todavia es incompleta.

Evidencia:

- `acelerador_panel/fronten/lib/tutor_grupos_service.php` sigue usando `mysqli` directo

`acelerador_panel/backend/docs/03-integracion-bd-frontend.md` documenta una migracion progresiva hacia REST

Por tanto:

- el frontend actual no consume todavia de forma generalizada una capa MCP;

tampoco consume aun de forma completa el backend REST nuevo.

MCP respecto al flujo ANECA/documental

Aqui la situacion es parecida:

- el evaluador ANECA tiene ya un flujo documental funcionando;

pero ese flujo usa su propio `Pipeline.php`, no `mcp-server`.

En terminos practicos:

- existe una idea fuerte de MCP como capa intermedia comun;

pero el repositorio aun muestra coexistencia de varias rutas documentales.

Sobre la separacion por rama o modulo

No se ha encontrado evidencia clara de una rama git especifica dedicada solo a MCP dentro del estado local actual.

Lo que si se observa con claridad es:

- un modulo separado `mcp-server/`;

documentacion que lo trata como pieza autonoma;

- y decisiones de integracion progresiva con backend y frontend.
-
- Por eso la formulacion mas fiel hoy es:
-
- `MCP esta implementado como modulo separado y preparado para integrarse, pero todavia no es la ruta unica ni plenamente integrada en toda la arquitectura de Acelerador.`
-
- ### 2.11 Principios reutilizables para otros proyectos
-

Separar fuente y negocio. El PDF, la DB o una API no deberian condicionar directamente la logica de evaluacion.

- Crear adaptadores por fuente. Cada origen puede tener su extractor propio mientras entregue una salida comun.

Definir contratos estables. El consumidor debe depender de JSON versionado, no del documento original.

- Mantener pipelines intercambiables. Si cambia el extractor, el resto del sistema no deberia romperse.

Soportar crecimiento incremental. Es valido empezar con una integracion parcial si la frontera entre modulos esta clara.

- Aceptar enriquecimiento posterior. Un JSON base puede fusionarse con datos manuales o con otras fuentes sin rehacer el pipeline completo.

Preparar extension sin obligarla desde el dia uno. El backend de tutorias muestra bien este principio con el publisher nulo.

2.12 Buenas practicas y riesgos

Buenas practicas que conviene mantener

- Mantener JSON como contrato intermedio entre extraccion y negocio.

Documentar siempre si un contrato es provisional, estable o pendiente de schema formal.

- Aislar la deteccion de PDF/OCR de la logica de negocio.

Dejar trazabilidad de pruebas, como ya se hace en `mcp-server/resultados/` y `docs/`.

- Permitir completado manual cuando la heuristica documental no pueda cerrar el dato con seguridad.
-
- **Riesgos y acoplamientos a evitar**
-

Acoplar evaluacion o frontend al contenido bruto del PDF.

- Mantener varios contratos JSON parecidos sin nombrarlos y versionarlos.

Duplicar pipelines sin declarar cual es el canonico.

- Depender de rutas Windows hardcodeadas para herramientas externas.

Asumir que "hay OCR" porque exista una clase, aunque esa clase no este conectada al flujo real.

- Mezclar informacion implementada y futura en la misma redaccion sin marcar el estado.
-
- **2.13 Aprendizajes clave**
-
- El proyecto ya deja varios aprendizajes utiles para el equipo:
-

La extraccion documental no es solo "leer PDFs"; es disenar una frontera entre evidencia y negocio.

- JSON aporta valor no solo por serializar datos, sino porque permite que distintos modulos hablen un lenguaje comun.

Un contrato intermedio hace posible validar antes la arquitectura, aunque no todas las integraciones esten cerradas.

- El completado manual no contradice la automatizacion; en dominios complejos como ANECA, la complementa.

Separar MCP como modulo propio tiene sentido cuando se prevé crecimiento a nuevas fuentes y nuevos consumidores.

2.14 Resumen ejecutivo final

Acelerador ya dispone de una base real para tratar documentos fuente sin acoplar el negocio al formato original. Esa base no esta unificada todavia en una sola ruta, pero si muestra una direccion arquitectonica consistente:

- MCP como capa intermedia de extraccion y normalizacion multi-fuente;

ANECA como flujo academico ya operativo con JSON intermedio real;

- backend modular validado primero por su cuenta;

y una integracion final aun progresiva entre fuente, backend y frontend.

La prioridad tecnica no es "reinventar" lo ya hecho, sino converger:

- sobre contratos mas claros,

sobre una ruta canonica de integracion,

- y sobre esquemas versionados que permitan reutilizar este enfoque en otros desarrollos.
-

3. Observaciones del analisis

-

Posibles incoherencias entre documentacion y codigo

-

`MCP_Documentacion.docx` no es un DOCX valido; hoy contiene texto plano con extension incorrecta.

- `mcp-server/config.json` declara `"extractor": "pdfplumber"`, pero el codigo real usa PHP con `smalot/pdfparser`, `pdftoppm` y `tesseract`.

`meritos/scraping/src/AnecaExtractor.php` devuelve el contrato simple tipo factura, no el esquema academico rico de `acelerador_evaluador_ANECA`.

- `acelerador_evaluador_ANECA/src/OcrProcessor.php` existe, pero `src/Pipeline.php` no lo usa en el flujo principal.

En salidas de `acelerador_evaluador_ANECA`, `metadatos_extraccion.comite` aparece fijado a `CSYJ` desde el extractor comun, incluso cuando existen modulos para otras areas.

- En esos mismos JSON, `metadatos_extraccion.archivo_pdf` va a `null`, mientras que `archivo_pdf` top-level si se rellena.
-
- **Partes poco documentadas o que convendria ampliar**
-

Cual debe ser la ruta canonica a medio plazo: `mcp-server`, `acelerador_evaluador_ANECA`, o una unificacion entre ambas.

- Versionado oficial de contratos JSON y `JSON Schema`.

Reglas de matching futuras por `ORCID`, `DOI` y `rama`.

- Politica de integracion de nuevas fuentes externas.

Criterios para saber cuando usar solo extraccion automatica y cuando exigir revision manual.

Riesgos de mantenimiento

- Coexistencia de varios pipelines con responsabilidad parecida.

Dependencias de herramientas externas con rutas locales hardcodeadas.

- Documentacion historica con problemas de codificacion de caracteres.

Falta de parametrizacion por area/comite en el extractor comun ANECA.

- Frontend y backend aun conviviendo con accesos directos a BD y contratos REST en paralelo.
-
- **Recomendaciones para mejorar sin perder lo ya construido**
-

Mantener `MCP_Documentacion.md` como documento base consolidado y no volver a fragmentar la narrativa.

- Corregir o retirar `MCP_Documentacion.docx` si no va a mantenerse como DOCX real.

Alinear `mcp-server/config.json` con la implementacion real.

- Declarar explicitamente cual es el contrato JSON canonico para cada flujo:

extraccion generica MCP

- extraccion academica ANECA

Publicar un `JSON Schema` versionado al menos para los contratos ya usados en integracion.

- Parametrizar el extractor ANECA por area/comite y revisar los metadatos generados.

Decidir si `meritos/scraping` se conserva como pipeline legado, como laboratorio o como base de convergencia, y documentarlo.

4. Validacion realizada para esta consolidacion

Contraste manual realizado sobre documentacion y codigo en:

- `mcp-server/`

`acelerador_evaluador_ANECA/`

- `acelerador_panel/backend/`

`docs/`

- raiz del repositorio
-
- Pruebas ejecutadas durante esta revision:

`php mcp-server/tests/unit_extract_pdf.php` -> PASS (`13/13`)

- `php acelerador_evaluador_ANECA/tests/unit_src.php` -> PASS (`6/6`)
-
- Esto no demuestra integracion completa entre todos los modulos, pero si confirma que los dos contratos documentales principales revisados mantienen hoy su comportamiento interno esperado.
-